

# GOOGLE GEMMA 4

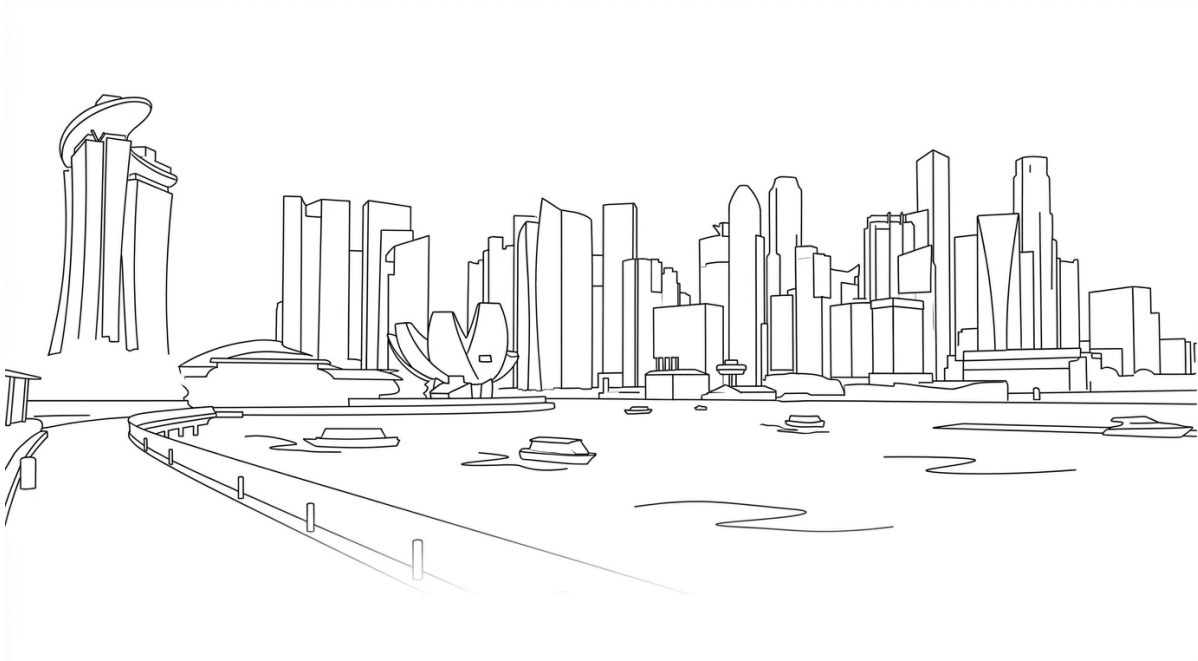
## Technical Summary

---

*Architecture, Licensing, Deployment & Benchmarks*

**Version** 2.0 (Revised with Model Card & p-RoPE Analysis)  
**Date** 3 April 2026  
**Classification** TLP:CLEAR — No restrictions  
**Author** Douglas Mun assisted by AI  
**Document Type** Technical Product Release Summary

© 2026 Douglas Mun. Licensed under CC BY 4.0 unless otherwise noted.



## Disclaimer

---

This document is an independent technical summary compiled from publicly available sources for informational purposes only. It is not affiliated with, endorsed by, or sponsored by Google, Google DeepMind, NVIDIA, or any other entity mentioned herein. All trademarks, product names, and company names referenced are the property of their respective owners.

The technical details, benchmark figures, and architectural descriptions contained in this document are based on information available as of the publication date. They may be subject to change as additional documentation (including formal technical reports) is released by the respective vendors. Readers are advised to verify critical details against primary sources before making deployment or procurement decisions.

This document was compiled with AI assistance. While every effort has been made to ensure accuracy through cross-referencing multiple independent sources, the authors make no warranties regarding completeness or fitness for any particular purpose.

**TLP: CLEAR** — This document may be distributed without restriction under the Traffic Light Protocol.

## Executive Summary

On 2 April 2026, Google DeepMind released Gemma 4, a family of four open-weight multimodal models derived from the same research underpinning the Gemini 3 model line. Gemma 4 is the first release in the Gemma series to adopt the Apache License 2.0, replacing the previous custom Gemma license that imposed usage restrictions and permitted unilateral term modifications by Google.

The family spans four model sizes — from sub-3B edge models to a 31B dense model — supporting text, image, and audio modalities with context windows up to 256K tokens. All variants are available via Hugging Face, Kaggle, and Ollama with day-one support across major inference frameworks, including llama.cpp, vLLM, MLX, and Ollama.

## 1. Licensing Change: Apache 2.0

Prior Gemma releases (Gemma 1 through Gemma 3) shipped under a proprietary “Gemma Terms of Use” license that restricted certain commercial use cases, included “Harmful Use” carve-outs that required legal interpretation, and allowed Google to modify the terms post-release. Notably, the Gemma source code (e.g., `gemma.cpp`) was already under Apache 2.0, but the **model weights** — the commercially relevant artefact — were governed by the restrictive custom license.

Gemma 4 moves both the weights and the code under the OSI-approved Apache License 2.0. Multiple independent sources confirm this applies without residual restrictions: no monthly active user caps, no acceptable-use policy enforcement clauses, no “Restricted Topics” carve-outs, and no restrictions on redistribution or commercial deployment. The LICENSE file in the google-deepmind/gemma repository contains the standard Apache 2.0 text with no addenda. This aligns Gemma’s licensing posture with competing open-weight model families, including Alibaba’s Qwen, Mistral, and Arcee, removing a significant friction point for enterprise legal review.

## 2. Model Family Architecture

Gemma 4 comprises four models organised into two deployment tiers. All models share a 262K-token vocabulary. Specifications below are from the official Google Gemma 4 model card.

### 2.1 Dense Models

| Property       | E2B                | E4B                | 31B Dense    |
|----------------|--------------------|--------------------|--------------|
| Total Params   | 5.1B (2.3B eff.)   | 8B (4.5B eff.)     | 30.7B        |
| Layers         | 35                 | 42                 | 60           |
| Sliding Window | 512 tokens         | 512 tokens         | 1024 tokens  |
| Context Length | 128K tokens        | 128K tokens        | 256K tokens  |
| Modalities     | Text, Image, Audio | Text, Image, Audio | Text, Image  |
| Vision Encoder | ~150M params       | ~150M params       | ~550M params |
| Audio Encoder  | ~300M params       | ~300M params       | None         |

### 2.2 Mixture-of-Experts (MoE) Model

|                   |             |
|-------------------|-------------|
| Total Parameters  | 25.2B       |
| Active Parameters | 3.8B        |
| Layers            | 30          |
| Sliding Window    | 1024 tokens |
| Context Length    | 256K tokens |

|                |                                           |
|----------------|-------------------------------------------|
| Expert Config  | 128 total / 8 active + 1 shared per token |
| Modalities     | Text, Image                               |
| Vision Encoder | ~550M params                              |

Each layer runs a dense GeGLU FFN in parallel with the 128-expert MoE block, and the outputs are summed and scaled by  $1/\sqrt{2}$ . Selective activation enables compute throughput comparable to a 4B model; however, all 25.2B parameters must remain resident in memory.

2.3 Proportional RoPE (p-RoPE) — The Mathematics of Long Context

**The Problem: Mechanical Amnesia at Scale.** Standard RoPE encodes position by rotating embedding vectors at different frequencies across dimensions. High-frequency dimensions encode precise local position; low-frequency dimensions encode broad semantic relationships. At 256K context, even the slowest-rotating dimensions accumulate enough angular displacement to wrap around and point backwards. The dot product between semantically related tokens separated by hundreds of thousands of positions becomes negative — the model perceives related concepts as mathematical opposites. This is described in Theorem 6.1 as a “geometric trap” in which long-distance semantic alignment fails via phase inversion rather than gradual degradation.

**The Solution: Freezing the Semantic Channels.** Proportional RoPE fixes this by truncating the slowest rotations to zero. In Gemma 4’s global attention layers, only 25% of embedding dimensions receive rotary encoding (partial=0.25). The remaining 75% are frozen at zero rotation — they never rotate regardless of token distance.

| Channel    | Dimensions      | Rotation             | Function                                                |
|------------|-----------------|----------------------|---------------------------------------------------------|
| Positional | 25% of head_dim | RoPE ( $\theta=1M$ ) | “Where is this token?” — positional awareness           |
| Semantic   | 75% of head_dim | Frozen (zero)        | “What is this token about?” — distance-agnostic meaning |

The semantic channel dimensions produce identical dot products whether tokens are 10 or 250,000 positions apart. They are pure content-matching channels that never suffer phase degradation.

**Why p-RoPE only applies to global layers:** Sliding-window layers (5 of every 6) attend within 512–1024 tokens, where standard RoPE ( $\theta=10K$ ) works perfectly. Global layers (1 of every 6) attend across the full 256K context and require p-RoPE to avoid the geometric trap. The final layer is always global, ensuring full visibility of context at the model’s last computation.

2.4 Additional Architectural Features

**Unified Keys and Values (K=V Weight Sharing).** In global attention layers, the V projection is eliminated; unified key-value tensors reduce both compute and memory for long-context generation.

**Shared KV Cache.** The last N layers (num\_kv\_shared\_layers) reuse K and V tensors from the last non-shared layer of the same attention type. In E2B, 20 of 35 layers share KV states.

**Per-Layer Embeddings (PLE).** Edge models incorporate a second embedding table mapping tokens to (layers × 256) dimensions. Each decoder layer receives a unique 256-dim slice, gated and added as a residual.

3. Capabilities

**Configurable Thinking Mode.** All Gemma 4 models include a built-in reasoning mode activated by including the <|think|> token at the start of the system prompt. When enabled, the model outputs internal reasoning within <|channel>thought tags before the final answer. Google has not disclosed whether this is backed by RL (e.g., GRPO) or by distillation from Gemini 3.

**Agentic Workflows.** Native function calling with structured JSON output and native system prompt support (standard system/assistant/user roles) enables building autonomous agents without grammar-constrained generation.

**Multimodal Input.** All models process text and images with variable aspect ratios and configurable visual token budgets (70, 140, 280, 560, 1120 tokens per image). Edge-tier models additionally support native audio input (up to 30 seconds) via a USM-style conformer encoder.

**Additional Capabilities.** Code generation and correction, interleaved multimodal input, over 140 languages natively, and GUI element detection with bounding box output in JSON format.

## 4. Local Inference Quick-Start Guide

### 4.1 Hardware Requirements

**Critical caveat:** The figures below represent model weights only and do not include the KV cache, which scales linearly with context length. At 256K context on the 31B model, the KV cache alone can consume 10–15 GB or more. A consumer GPU with 24 GB VRAM running the 31B at Q4 will likely OOM well before reaching the full 256K context window.

| Model         | 4-bit (Q4_K_M) | 8-bit (Q8_0) | Full Precision (BF16) |
|---------------|----------------|--------------|-----------------------|
| E2B           | ~3 GB          | ~5 GB        | ~15 GB                |
| E4B           | ~4 GB          | ~5 GB        | ~15 GB                |
| 26B A4B (MoE) | ~18 GB         | ~28 GB       | Single 80 GB H100     |
| 31B Dense     | ~20 GB         | ~34 GB       | Single 80 GB H100     |

### 4.2 Quick-Start: llama.cpp

llama.cpp requires build b8636 or later for full support for Gemma 4.

#### Step 1 — Install or update llama.cpp

##### macOS (Homebrew):

```
brew upgrade llama.cpp
# If latest build unavailable:
brew install llama.cpp --HEAD
```

##### Linux / Build from source:

```
git clone https://github.com/ggml-org/llama.cpp
cmake llama.cpp -B llama.cpp/build -DBUILD_SHARED_LIBS=OFF -DGGML_CUDA=ON
cmake --build llama.cpp/build --config Release -j --clean-first
```

For Apple Silicon (Metal), set `-DGGML_CUDA=OFF`; Metal acceleration is enabled by default. For AMD GPUs, use `-DGGML_HIP=ON`.

#### Step 2 — Select and run by hardware profile

| Hardware         | Recommended Command                                                   |
|------------------|-----------------------------------------------------------------------|
| MacBook 16 GB    | <code>llama-server -hf ggml-org/gemma-4-E4B-it-GGUF:Q8_0</code>       |
| MacBook 24 GB+   | <code>llama-server -hf ggml-org/gemma-4-26B-A4B-it-GGUF:Q4_K_M</code> |
| RTX 3090 (24 GB) | <code>llama-server -hf ggml-org/gemma-4-26B-A4B-it-GGUF:Q4_K_M</code> |
| RTX 5090 (32 GB) | <code>llama-server -hf ggml-org/gemma-4-26B-A4B-it-GGUF:Q8_0</code>   |
| H100 (80 GB)     | <code>llama-server -hf ggml-org/gemma-4-31B-it-GGUF:F16</code>        |

The `llama-server` command automatically downloads the GGUF checkpoint and starts an OpenAI-compatible API endpoint at `http://localhost:8080/v1`.

### 4.3 Quick-Start: Ollama

Ollama provides the fastest path to running Gemma 4 locally with minimal configuration. Download from <https://ollama.com>.

```
ollama run gemma4          # Default (auto-selects best variant)
ollama run gemma4:e4b      # Edge – laptops with 8+ GB RAM
ollama run gemma4:26b      # MoE – consumer GPUs with 24+ GB
ollama run gemma4:31b      # Dense – workstations with 32+ GB
```

### 4.4 Quick-Start: MLX (Apple Silicon)

```
curl -fsSL https://raw.githubusercontent.com/unslothai/unsloth/
refs/heads/main/install_gemma4mlx.sh | sh
source ~/.unsloth/unsloth_gemma4mlx/bin/activate
python -m mlx_lm chat --model unsloth/gemma-4-E4B-it-UD-MLX-4bit
```

### 4.5 Quick-Start: Python (Hugging Face Transformers)

```
from transformers import pipeline
pipe = pipeline("any-to-any", model="google/gemma-4-e4b-it")
result = pipe({"text": "Explain quantum computing."})
```

### 4.6 Recommended Inference Parameters

Google's default sampling configuration: temperature 1.0, top-p 0.95, top-k 64. Repetition and presence penalties should remain at 1.0 unless token repetition is observed. To enable chain-of-thought reasoning, include `<|think|>` at the start of the system prompt.

### 4.7 Connecting to Agent Frameworks

Both llama-server and Ollama expose OpenAI-compatible API endpoints, enabling direct integration with agent frameworks without custom adapters. Verified compatible frameworks include OpenClaw, Hermes, Pi, and Open Code.

### 4.8 Additional Supported Runtimes

Day-one support: vLLM, mistral.rs (Rust-native), SGLang, NVIDIA NIM, LM Studio, Unsloth Studio, Keras, Baseten, Docker, MaxText, and transformers.js (browser inference via WebGPU). AMD GPUs supported via ROCm through vLLM, SGLang, llama.cpp, and Lemonade Server.

## 5. Benchmark Results (Official)

All results below are for instruction-tuned models, sourced from the official Gemma 4 model card.

### 5.1 Text Benchmarks

| Benchmark        | 31B   | 26B MoE | E4B   | E2B   | Gemma 3 27B |
|------------------|-------|---------|-------|-------|-------------|
| MMLU Pro         | 85.2% | 82.6%   | 69.4% | 60.0% | 67.6%       |
| AIME 2026        | 89.2% | 88.3%   | 42.5% | 37.5% | 20.8%       |
| LiveCodeBench v6 | 80.0% | 77.1%   | 52.0% | 44.0% | 29.1%       |
| Codeforces Elo   | 2150  | 1718    | 940   | 633   | 110         |
| GPQA Diamond     | 84.3% | 82.3%   | 58.6% | 43.4% | 42.4%       |
| BigBench XH      | 74.4% | 64.8%   | 33.1% | 21.9% | 19.3%       |
| MMMLU            | 88.4% | 86.3%   | 76.6% | 67.4% | 70.7%       |

## 5.2 Vision Benchmarks

| Benchmark     | 31B   | 26B MoE | E4B   | E2B   | Gemma 3 27B |
|---------------|-------|---------|-------|-------|-------------|
| MMMU Pro      | 76.9% | 73.8%   | 52.6% | 44.2% | 49.7%       |
| MATH-Vision   | 85.6% | 82.4%   | 59.5% | 52.4% | 46.0%       |
| MedXPertQA MM | 61.3% | 58.1%   | 28.7% | 23.5% | —           |

The 31B’s BigBench Extra Hard score of 74.4% represents a nearly 4× improvement over Gemma 3 27B’s 19.3%. On the Arena AI text leaderboard, the 31B ranked #3 (Elo 1452) and the 26B MoE ranked #6 (Elo 1441).

## 6. Prompt Speculative Decoding (300 t/s Demo)

On 2 April 2026, Georgi Gerganov (creator of llama.cpp) demonstrated Gemma 4 26B A4B Q8\_0 running on a Mac Studio M2 Ultra (3-year-old hardware) with llama.cpp’s built-in WebUI, MCP support (web-search, HuggingFace, GitHub tools), and **prompt speculative decoding** — achieving 300 tokens/second in a real-time video.

**How it works:** This is distinct from traditional two-model speculative decoding. When llama.cpp’s MCP integration injects tool results (thousands of tokens of search results) into the prompt, and the model then generates a response referencing that content, llama.cpp proposes candidate token sequences drawn from the prompt itself as “draft” tokens, then batch-verifies them in a single forward pass. Since MCP tool results are often quoted or paraphrased, the acceptance rate is extremely high.

**Critical caveat:** The 300 t/s represents peak burst throughput during prompt-recitation phases. For purely novel token generation, throughput drops to approximately 30–60 t/s on that hardware. Gerganov posted a follow-up caveat acknowledging the prompt-recitation component.

**Significance for agentic workflows:** This is particularly relevant for RAG and CTI workflows where the model ingests retrieved content via MCP tools and then produces structured reports — the inherent “retrieve → summarise” pattern maximises prompt speculative decoding acceptance rates.

## 7. Competitive Context

Gemma 4 enters a competitive open-weight landscape. Over the preceding year, Chinese open-source model families — notably Alibaba’s Qwen, DeepSeek, and GLM — gained significant market share, with Qwen overtaking Meta’s Llama as the most-used self-hosted model globally by late 2025.

| Model             | Arena Elo | GPQA Diamond | MMLU Pro | Active Params |
|-------------------|-----------|--------------|----------|---------------|
| Gemma 4 31B Dense | 1452 (#3) | 85.7%        | 85.2%    | 31B           |
| Gemma 4 26B MoE   | 1441 (#6) | 79.2%        | —        | 3.8B          |
| Qwen 3.5 27B      | —         | 85.8%        | —        | 27B           |
| Llama 4 Scout     | —         | —            | —        | 17B (10M ctx) |

Google has indicated that additional Gemma 4 model sizes may follow the initial release. The E2B and E4B models serve as the foundation for Gemini Nano 4, Google’s next-generation on-device model for Android, expected on consumer devices later in 2026.

## 8. Open Technical Questions

**Training methodology for reasoning.** Google has not disclosed whether the thinking/reasoning capability uses RL (GRPO/PPO), distillation from Gemini 3, or another approach. Preserving reasoning capability during SFT requires mixing reasoning-style and direct-answer examples (Unsloth recommends a minimum of 75% reasoning-style examples).

**Speculative decoding.** No official draft-model pairing has been announced (e.g., using E2B as a drafter for the 31B). Community experimentation is expected.

**Tokeniser compatibility.** Google has not confirmed whether Gemma 4 uses the same tokeniser as Gemma 3 (the Gemini 2.0 tokeniser with a 256K vocabulary). If changed, existing vector database embeddings would require full re-indexing for RAG pipelines.

**Full technical report.** As of 3 April 2026, no Gemma 4 technical report has been published. The Gemma 3 Technical Report (arXiv:2503.19786) remains the most recent. Google I/O 2026 (expected in May) may provide additional details.

## 9. Model Weights & Download Links

| Source                            | URL                                                                                                                           |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Hugging Face (Official)           | <a href="https://huggingface.co/collections/google/gemma-4">https://huggingface.co/collections/google/gemma-4</a>             |
| Hugging Face (GGUF — Unsloth)     | <a href="https://huggingface.co/collections/unsloth/gemma-4-gguf">https://huggingface.co/collections/unsloth/gemma-4-gguf</a> |
| Kaggle                            | <a href="https://www.kaggle.com/models/google/gemma-4">https://www.kaggle.com/models/google/gemma-4</a>                       |
| Ollama                            | <a href="https://ollama.com/library/gemma4">https://ollama.com/library/gemma4</a>                                             |
| Google AI Studio (31B, 26B)       | <a href="https://aistudio.google.dev">https://aistudio.google.dev</a>                                                         |
| Google AI Edge Gallery (E4B, E2B) | Via Android app                                                                                                               |

## Sources

- [1] Google AI for Developers, “Gemma 4 Model Card,” 2 Apr 2026[https://ai.google.dev/gemma/docs/core/model\\_card\\_4](https://ai.google.dev/gemma/docs/core/model_card_4)
- [2] Google DeepMind, “Gemma 4: Byte for byte, the most capable open models,” 2 Apr 2026<https://blog.google/innovation-and-ai/technology/developers-tools/gemma-4/>
- [3] Google Open Source Blog, “Gemma 4: Expanding the Gemmaverse with Apache 2.0,” 2 Apr 2026<https://opensource.googleblog.com/2026/03/gemma-4-expanding-the-gemmaverse-with-apache-20.html>
- [4] “Gemma 4: Google’s Open Source Leap and p-RoPE Integration” (video), YouTube<https://www.youtube.com/watch?v=iRbZYUJiecl>
- [5] Hugging Face, “Welcome Gemma 4: Frontier multimodal intelligence on device,” 2 Apr 2026<https://huggingface.co/blog/gemma4>
- [6] Georgi Gerganov (@ggerganov), llama.cpp Gemma 4 demo, 2 Apr 2026<https://x.com/ggerganov/status/2039752638384709661>
- [7] Latent Space, “AINews: Gemma 4,” 3 Apr 2026<https://www.latent.space/p/ainews-gemma-4-the-best-small-multimodal>
- [8] Google AI for Developers, “Thinking mode in Gemma,” 2 Apr 2026<https://ai.google.dev/gemma/docs/capabilities/thinking>
- [9] google-deepmind/gemma LICENSE file (Apache 2.0)<https://github.com/google-deepmind/gemma/blob/main/LICENSE>
- [10] NVIDIA Developer Blog, “Bringing AI Closer to the Edge and On-Device with Gemma 4,” 2 Apr 2026<https://developer.nvidia.com/blog/bringing-ai-closer-to-the-edge-and-on-device-with-gemma-4/>
- [11] NVIDIA Blog, “From RTX to Spark: NVIDIA Accelerates Gemma 4,” 2 Apr 2026<https://blogs.nvidia.com/blog/rtx-ai-garage-open-models-google-gemma-4/>
- [12] Unsloth Documentation, “Gemma 4 — How to Run Locally”<https://unsloth.ai/docs/models/gemma-4>
- [13] Ollama, “gemma4 Model Library”<https://ollama.com/library/gemma4>
- [14] Google Developers Blog, “Agentic skills at the edge with Gemma 4,” 2 Apr 2026<https://developers.googleblog.com/bring-state-of-the-art-agentic-skills-to-the-edge-with-gemma-4/>
- [15] Gemma 4 Architecture — Complete Technical Comparison (community)<https://g4.sj5.pl/>
- [16] WaveSpeedAI, “What Is Google Gemma 4?” 3 Apr 2026<https://wavespeed.ai/blog/posts/what-is-google-gemma-4/>
- [17] VentureBeat, “Google releases Gemma 4 under Apache 2.0,” 2 Apr 2026<https://venturebeat.com/technology/google-releases-gemma-4-under-apache-2-0-and-that-license-change-may-matter>
- [18] AMD, “Day 0 Support for Gemma 4 on AMD,” 2 Apr 2026<https://www.amd.com/en/developer/resources/technical-articles/2026/day-0-support-for-gemma-4-on-amd-processors-and-gpus.html>
- [19] Gemma 3 Technical Report, arXiv:2503.19786, Google DeepMind, Mar 2025<https://arxiv.org/abs/2503.19786>
- [20] Apache Software Foundation, “Apache License, Version 2.0”<https://www.apache.org/licenses/LICENSE-2.0>

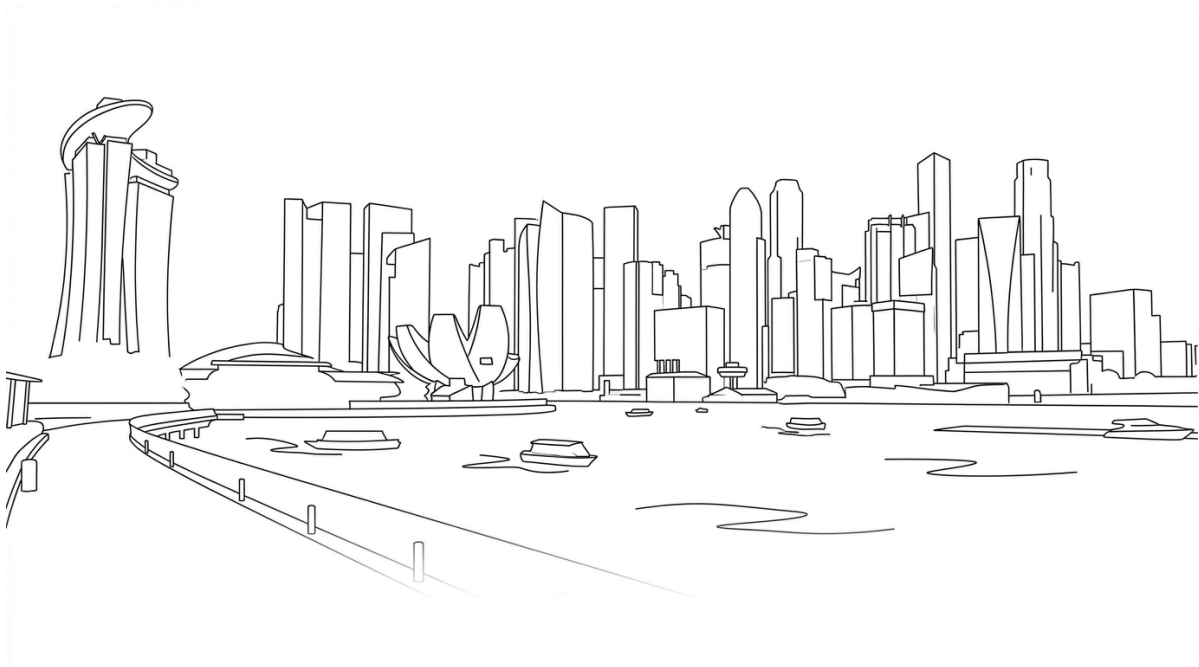


# Google Gemma 4: Technical Summary

---

Version 2.0 | 3 April 2026 | TLP:CLEAR  
Douglas Mun assisted by AI

© 2026 Douglas Mun. Licensed under CC BY 4.0 unless otherwise noted.



**Gemma 4** released with text, audio and image input and long up to 256K context window! [Learn more](#)  
(/gemma/docs/core)

## Gemma 4 model card



[Hugging Face](https://huggingface.co/collections/google/gemma-4) (https://huggingface.co/collections/google/gemma-4) | [GitHub](https://github.com/google-gemma) (https://github.com/google-gemma) | [Launch Blog](#) (https://blog.google/innovation-and-ai/technology/developers-tools/gemma-4/) | [Documentation](#) (https://ai.google.dev/gemma/docs/core)

**License:** [Apache 2.0](https://ai.google.dev/gemma/docs/gemma_4_license) (https://ai.google.dev/gemma/docs/gemma\_4\_license) | **Authors:** [Google DeepMind](https://deepmind.google/models/gemma/) (https://deepmind.google/models/gemma/)

Gemma is a family of open models built by Google DeepMind. Gemma 4 models are multimodal, handling text and image input (with audio supported on small models) and generating text output. This release includes open-weights models in both pre-trained and instruction-tuned variants. Gemma 4 features a context window of up to 256K tokens and maintains multilingual support in over 140 languages.

Featuring both Dense and Mixture-of-Experts (MoE) architectures, Gemma 4 is well-suited for tasks like text generation, coding, and reasoning. The models are available in four distinct sizes: **E2B**, **E4B**, **26B A4B**, and **31B**. Their diverse sizes make them deployable in environments ranging from high-end phones to laptops and servers, democratizing access to state-of-the-art AI.

Gemma 4 introduces key **capability and architectural advancements**:

- **Reasoning** – All models in the family are designed as highly capable reasoners, with configurable thinking modes.
- **Extended Multimodalities** – Processes Text, Image with variable aspect ratio and resolution support (all models), Video, and Audio (featured natively on the E2B and E4B models).
- **Diverse & Efficient Architectures** – Offers Dense and Mixture-of-Experts (MoE) variants of different sizes for scalable deployment.
- **Optimized for On-Device** – Smaller models are specifically designed for efficient local execution on laptops and mobile devices.
- **Increased Context Window** – The small models feature a 128K context window, while the medium models support 256K.
- **Enhanced Coding & Agentic Capabilities** – Achieves notable improvements in coding benchmarks alongside native function-calling support, powering highly capable autonomous agents.
- **Native System Prompt Support** – Gemma 4 introduces native support for the system role, enabling more structured and controllable conversations.

# Models Overview

Gemma 4 models are designed to deliver frontier-level performance at each size, targeting deployment scenarios from mobile and edge devices (E2B, E4B) to consumer GPUs and workstations (26B A4B, 31B). They are well-suited for reasoning, agentic workflows, coding, and multimodal understanding.

The models employ a hybrid attention mechanism that interleaves local sliding window attention with full global attention, ensuring the final layer is always global. This hybrid design delivers the processing speed and low memory footprint of a lightweight model without sacrificing the deep awareness required for complex, long-context tasks. To optimize memory for long contexts, global layers feature unified Keys and Values, and apply Proportional RoPE (p-RoPE).

## Dense Models

| Property                  | E2B                                   | E4B                                 | 31B Dense   |
|---------------------------|---------------------------------------|-------------------------------------|-------------|
| Total Parameters          | 2.3B effective (5.1B with embeddings) | 4.5B effective (8B with embeddings) | 30.7B       |
| Layers                    | 35                                    | 42                                  | 60          |
| Sliding Window            | 512 tokens                            | 512 tokens                          | 1024 tokens |
| Context Length            | 128K tokens                           | 128K tokens                         | 256K tokens |
| Vocabulary Size           | 262K                                  | 262K                                | 262K        |
| Supported Modalities      | Text, Image, Audio                    | Text, Image, Audio                  | Text, Image |
| Vision Encoder Parameters | ~150M                                 | ~150M                               | ~550M       |
| Audio Encoder Parameters  | ~300M                                 | ~300M                               | No Audio    |

The "E" in E2B and E4B stands for "effective" parameters. The smaller models incorporate Per-Layer Embeddings (PLE) to maximize parameter efficiency in on-device deployments. Rather than adding more layers or parameters to the model, PLE gives each decoder layer its own small embedding for every token. These embedding tables are large but are only used for quick lookups, which is why the effective parameter count is much smaller than the total.

## Mixture-of-Experts (MoE) Model

| Property          | 26B A4B MoE |
|-------------------|-------------|
| Total Parameters  | 25.2B       |
| Active Parameters | 3.8B        |
| Layers            | 30          |
|                   |             |

| Property                  | 26B A4B MoE                       |
|---------------------------|-----------------------------------|
| Sliding Window            | 1024 tokens                       |
| Context Length            | 256K tokens                       |
| Vocabulary Size           | 262K                              |
| Expert Count              | 8 active / 128 total and 1 shared |
| Supported Modalities      | Text, Image                       |
| Vision Encoder Parameters | ~550M                             |

The "A" in 26B A4B stands for "active parameters" in contrast to the total number of parameters the model contains. By only activating a 4B subset of parameters during inference, the Mixture-of-Experts model runs much faster than its 26B total might suggest. This makes it an excellent choice for fast inference compared to the dense 31B model since it runs almost as fast as a 4B-parameter model.

## Benchmark Results

These models were evaluated against a large collection of different datasets and metrics to cover different aspects of text generation. Evaluation results marked in the table are for instruction-tuned models.

|                       | Gemma 4<br>31B | Gemma 4 26B<br>A4B | Gemma 4<br>E4B | Gemma 4<br>E2B | Gemma 3 27B (no<br>think) |
|-----------------------|----------------|--------------------|----------------|----------------|---------------------------|
| MMLU Pro              | 85.2%          | 82.6%              | 69.4%          | 60.0%          | 67.6%                     |
| AIME 2026 no tools    | 89.2%          | 88.3%              | 42.5%          | 37.5%          | 20.8%                     |
| LiveCodeBench v6      | 80.0%          | 77.1%              | 52.0%          | 44.0%          | 29.1%                     |
| Codeforces ELO        | 2150           | 1718               | 940            | 633            | 110                       |
| GPQA Diamond          | 84.3%          | 82.3%              | 58.6%          | 43.4%          | 42.4%                     |
| Tau2 (average over 3) | 76.9%          | 68.2%              | 42.2%          | 24.5%          | 16.2%                     |
| HLE no tools          | 19.5%          | 8.7%               | -              | -              | -                         |
| HLE with search       | 26.5%          | 17.2%              | -              | -              | -                         |
| BigBench Extra Hard   | 74.4%          | 64.8%              | 33.1%          | 21.9%          | 19.3%                     |
|                       |                |                    |                |                |                           |

|                                                           | Gemma 4<br>31B | Gemma 4 26B<br>A4B | Gemma 4<br>E4B | Gemma 4<br>E2B | Gemma 3 27B (no<br>think) |
|-----------------------------------------------------------|----------------|--------------------|----------------|----------------|---------------------------|
| MMMLU                                                     | 88.4%          | 86.3%              | 76.6%          | 67.4%          | 70.7%                     |
| Vision                                                    |                |                    |                |                |                           |
| MMMU Pro                                                  | 76.9%          | 73.8%              | 52.6%          | 44.2%          | 49.7%                     |
| OmniDocBench 1.5 (average edit distance, lower is better) | 0.131          | 0.149              | 0.181          | 0.290          | 0.365                     |
| MATH-Vision                                               | 85.6%          | 82.4%              | 59.5%          | 52.4%          | 46.0%                     |
| MedXPertQA MM                                             | 61.3%          | 58.1%              | 28.7%          | 23.5%          | -                         |
| Audio                                                     |                |                    |                |                |                           |
| CoVoST                                                    | -              | -                  | 35.54          | 33.47          | -                         |
| FLEURS (lower is better)                                  | -              | -                  | 0.08           | 0.09           | -                         |
| Long Context                                              |                |                    |                |                |                           |
| MRCR v2 8 needle 128k (average)                           | 66.4%          | 44.1%              | 25.4%          | 19.1%          | 13.5%                     |

## Core Capabilities

Gemma 4 models handle a broad range of tasks across text, vision, and audio. Key capabilities include:

- **Thinking** – Built-in reasoning mode that lets the model think step-by-step before answering.
- **Long Context** – Context windows of up to 128K tokens (E2B/E4B) and 256K tokens (26B A4B/31B).
- **Image Understanding** – Object detection, Document/PDF parsing, screen and UI understanding, chart comprehension, OCR (including multilingual), handwriting recognition, and pointing. Images can be processed at variable aspect ratios and resolutions.
- **Video Understanding** – Analyze video by processing sequences of frames.
- **Interleaved Multimodal Input** – Freely mix text and images in any order within a single prompt.
- **Function Calling** – Native support for structured tool use, enabling agentic workflows.
- **Coding** – Code generation, completion, and correction.
- **Multilingual** – Out-of-the-box support for 35+ languages, pre-trained on 140+ languages.
- **Audio** (E2B and E4B only) – Automatic speech recognition (ASR) and speech-to-translated-text translation across multiple languages.

# Getting Started

You can use all Gemma 4 models with the latest version of Transformers. To get started, install the necessary dependencies in your environment:

```
pip install -U transformers torch accelerate
```

Once you have everything installed, you can proceed to load the model with the code below:

```
import torch
from transformers import AutoProcessor, AutoModelForCausalLM

MODEL_ID = "google/gemma-4-E2B-it"

# Load model
processor = AutoProcessor.from_pretrained(MODEL_ID)
model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID,
    dtype=torch.bfloat16,
    device_map="auto"
)
```

Once the model is loaded, you can start generating output:

```
# Prompt
messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Write a short joke about saving RAM."},
]

# Process input
text = processor.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True,
    enable_thinking=False
)
inputs = processor(text=text, return_tensors="pt").to(model.device)
input_len = inputs["input_ids"].shape[-1]

# Generate output
outputs = model.generate(**inputs, max_new_tokens=1024)
response = processor.decode(outputs[0][input_len:], skip_special_tokens=False)

# Parse thinking
processor.parse_response(response)
```

To enable reasoning, set `enable_thinking=True` and the `parse_response` function will take care of parsing the thinking output.

## Best Practices

For the best performance, use these configurations and best practices:

## 1. Sampling Parameters

Use the following standardized sampling configuration across all use cases:

- `temperature=1.0`
- `top_p=0.95`
- `top_k=64`

## 2. Thinking Mode Configuration

Compared to Gemma 3, the models use standard `system`, `assistant`, and `user` roles. To properly manage the thinking process, use the following control tokens:

- **Trigger Thinking:** Thinking is enabled by including the `<|think|>` token at the start of the system prompt. To disable thinking, remove the token.
- **Standard Generation:** When thinking is enabled, the model will output its internal reasoning followed by the final answer using this structure: `<|channel>thought\n[Internal reasoning]<channel|>`
- **Disabled Thinking Behavior:** For all models except for the E2B and E4B variants, if thinking is disabled, the model will still generate the tags but with an empty thought block: `<|channel>thought\n<channel|>[Final answer]`

Note that many libraries like Transformers and llama.cpp handle the complexities of the chat template for you.

## 3. Multi-Turn Conversations

- **No Thinking Content in History:** In multi-turn conversations, the historical model output should only include the final response. Thoughts from previous model turns must *not be added* before the next user turn begins.

## 4. Modality order

- For optimal performance with multimodal inputs, place image and/or audio content **before** the text in your prompt.

## 5. Variable Image Resolution

Aside from variable aspect ratios, Gemma 4 supports variable image resolution through a configurable visual token budget, which controls how many tokens are used to represent an image. A higher token budget preserves more visual detail at the cost of additional compute, while a lower budget enables faster inference for tasks that don't require fine-grained understanding.

- The supported token budgets are: **70, 140, 280, 560, and 1120**.
  - Use *lower budgets* for classification, captioning, or video understanding, where faster inference and processing many frames outweigh fine-grained detail.
  - Use *higher budgets* for tasks like OCR, document parsing, or reading small text.

## 6. Audio

Use the following prompt structures for audio processing:

- **Audio Speech Recognition (ASR)**

Transcribe the following speech segment in {LANGUAGE} into {LANGUAGE} text.

Follow these specific instructions for formatting the answer:

- \* Only output the transcription, with no newlines.
- \* When transcribing numbers, write the digits, i.e. write 1.7 and not one point seven, and write

- **Automatic Speech Translation (AST)**

Transcribe the following speech segment in {SOURCE\_LANGUAGE}, then translate it into {TARGET\_LANGUAGE}. When formatting the answer, first output the transcription in {SOURCE\_LANGUAGE}, then one newline,

## 7. Audio and Video Length

All models support image inputs and can process videos as frames whereas the E2B and E4B models also support audio inputs. Audio supports a maximum length of 30 seconds. Video supports a maximum of 60 seconds assuming the images are processed at one frame per second.

## Model Data

Data used for model training and how the data was processed.

### Training Dataset

Our pre-training dataset is a large-scale, diverse collection of data encompassing a wide range of domains and modalities, which includes web documents, code, images, audio, with a cutoff date of January 2025. Here are the key components:

- **Web Documents:** A diverse collection of web text ensures the model is exposed to a broad range of linguistic styles, topics, and vocabulary. The training dataset includes content in over 140 languages.
- **Code:** Exposing the model to code helps it to learn the syntax and patterns of programming languages, which improves its ability to generate code and understand code-related questions.
- **Mathematics:** Training on mathematical text helps the model learn logical reasoning, symbolic representation, and to address mathematical queries.
- **Images:** A wide range of images enables the model to perform image analysis and visual data extraction tasks.

The combination of these diverse data sources is crucial for training a powerful multimodal model that can handle a wide variety of different tasks and data formats.

### Data Preprocessing

Here are the key data cleaning and filtering methods applied to the training data:

- **CSAM Filtering:** Rigorous CSAM (Child Sexual Abuse Material) filtering was applied at multiple stages in the data preparation process to ensure the exclusion of harmful and illegal content.



- **Sensitive Data Filtering:** As part of making Gemma pre-trained models safe and reliable, automated techniques were used to filter out certain personal information and other sensitive data from training sets.
- **Additional methods:** Filtering based on content quality and safety in line with [our policies](https://ai.google/static/documents/ai-responsibility-update-published-february-2025.pdf) (<https://ai.google/static/documents/ai-responsibility-update-published-february-2025.pdf>).

## Ethics and Safety

As open models become central to enterprise infrastructure, provenance and security are paramount. Developed by Google DeepMind, Gemma 4 undergoes the same rigorous safety evaluations as our proprietary Gemini models.

## Evaluation Approach

Gemma 4 models were developed in partnership with internal safety and responsible AI teams. A range of automated as well as human evaluations were conducted to help improve model safety. These evaluations align with [Google's AI principles](https://ai.google/principles/) (<https://ai.google/principles/>), as well as safety policies, which aim to prevent our generative AI models from generating harmful content, including:

- Content related to child sexual abuse material and exploitation
- Dangerous content (e.g., promoting suicide, or instructing in activities that could cause real-world harm)
- Sexually explicit content
- Hate speech (e.g., dehumanizing members of protected groups)
- Harassment (e.g., encouraging violence against people)

## Evaluation Results

For all areas of safety testing, we saw major improvements in all categories of content safety relative to previous Gemma models. Overall, Gemma 4 models significantly outperform Gemma 3 and 3n models in improving safety, while keeping unjustified refusals low. All testing was conducted without safety filters to evaluate the model capabilities and behaviors. For both text-to-text and image-to-text, and across all model sizes, the model produced minimal policy violations, and showed significant improvements over previous Gemma models' performance.

## Usage and Limitations

These models have certain limitations that users should be aware of.

### Intended Usage

Multimodal models (capable of processing vision, language, and/or audio) have a wide range of applications across various industries and domains. The following list of potential uses is not comprehensive. The purpose of this list is to provide contextual information about the possible use-cases that the model creators considered as part of model training and development.

- **Content Creation and Communication**
  - **Text Generation:** These models can be used to generate creative text formats such as poems, scripts, code, marketing copy, and email drafts.

- **Chatbots and Conversational AI:** Power conversational interfaces for customer service, virtual assistants, or interactive applications.
- **Text Summarization:** Generate concise summaries of a text corpus, research papers, or reports.
- **Image Data Extraction:** These models can be used to extract, interpret, and summarize visual data for text communications.
- **Audio Processing and Interaction:** The smaller models (E2B and E4B) can analyze and interpret audio inputs, enabling voice-driven interactions and transcriptions.
- **Research and Education**
  - **Natural Language Processing (NLP) and VLM Research:** These models can serve as a foundation for researchers to experiment with VLM and NLP techniques, develop algorithms, and contribute to the advancement of the field.
  - **Language Learning Tools:** Support interactive language learning experiences, aiding in grammar correction or providing writing practice.
    - **Knowledge Exploration:** Assist researchers in exploring large bodies of text by generating summaries or answering questions about specific topics.

## Limitations

- **Training Data**
  - The quality and diversity of the training data significantly influence the model's capabilities. Biases or gaps in the training data can lead to limitations in the model's responses.
  - The scope of the training dataset determines the subject areas the model can handle effectively.
- **Context and Task Complexity**
  - Models perform well on tasks that can be framed with clear prompts and instructions. Open-ended or highly complex tasks might be challenging.
  - A model's performance can be influenced by the amount of context provided (longer context generally leads to better outputs, up to a certain point).
- **Language Ambiguity and Nuance**
  - Natural language is inherently complex. Models might struggle to grasp subtle nuances, sarcasm, or figurative language.
- **Factual Accuracy**
  - Models generate responses based on information they learned from their training datasets, but they are not knowledge bases. They may generate incorrect or outdated factual statements.
- **Common Sense**
  - Models rely on statistical patterns in language. They might lack the ability to apply common sense reasoning in certain situations.

## Ethical Considerations and Risks

The development of vision-language models (VLMs) raises several ethical concerns. In creating an open model, we have carefully considered the following:

- **Bias and Fairness**

- VLMs trained on large-scale, real-world text and image data can reflect socio-cultural biases embedded in the training material. Gemma 4 models underwent careful scrutiny, input data pre-processing, and post-training evaluations as reported in this card to help mitigate the risk of these biases.

- **Misinformation and Misuse**

- VLMs can be misused to generate text that is false, misleading, or harmful.
- Guidelines are provided for responsible use with the model, see the [Responsible Generative AI Toolkit](https://ai.google.dev/responsible) (<https://ai.google.dev/responsible>).

- **Transparency and Accountability**

- This model card summarizes details on the models' architecture, capabilities, limitations, and evaluation processes.
- A responsibly developed open model offers the opportunity to share innovation by making VLM technology accessible to developers and researchers across the AI ecosystem.

### **Risks identified and mitigations:**

- **Generation of harmful content:** Mechanisms and guidelines for content safety are essential. Developers are encouraged to exercise caution and implement appropriate content safety safeguards based on their specific product policies and application use cases.
- **Misuse for malicious purposes:** Technical limitations and developer and end-user education can help mitigate against malicious applications of VLMs. Educational resources and reporting mechanisms for users to flag misuse are provided.
- **Privacy violations:** Models were trained on data filtered for removal of certain personal information and other sensitive data. Developers are encouraged to adhere to privacy regulations with privacy-preserving techniques.
- **Perpetuation of biases:** It's encouraged to perform continuous monitoring (using evaluation metrics, human review) and the exploration of de-biasing techniques during model training, fine-tuning, and other use cases.

### **Benefits**

At the time of release, this family of models provides high-performance open vision-language model implementations designed from the ground up for responsible AI development compared to similarly sized models.

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-04-02 UTC.